



LUMS



Logistic Regression Classification

Agha Ali Raza



CS535/EE514 – Machine Learning

Sources

- **Machine Learning for Intelligent Systems**, Kilian Weinberger, Cornell, Video Lectures 35-37,
<https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote20.html>
- **Deep Learning Specialization**, Andrew Ng
https://www.coursera.org/specializations/deep-learning?utm_source=deeplearningai&utm_medium=institutions&utm_campaign=SocialYoutubeDLSC1W1L1
 - **Video Lectures: C1W3L1 – C1W4L6:**
<https://www.youtube.com/playlist?list=PLpFsSf5Dm-pd5d3rjNtIXUHT-v7bdaEle>
- **A beginner's guide to deriving and implementing backpropagation**, by Pranav Budhwant
<https://link.medium.com/Zp3zxNWpf6>
- **Tensorflow playground:** <https://playground.tensorflow.org/>
- **Machine Learning Playground:** <https://ml-playground.com/#>

Logistic Regression

- Discover the relation between features/cues/events and some outcome
- Logistic regression is the baseline supervised machine learning algorithm for classification
- It has a very close relationship with neural networks
 - A neural network can be viewed as a series of logistic regression classifiers stacked on top of each other

Logistic Regression

- The outcome is viewed as a weighted sum of features
- A bias (θ_0 , or y-intercept) to adjust the trend of the sum
- The learned weights enhance important (discriminating) features and suppress unimportant ones

Logistic Regression

- Logistic regression learns a vector of **weights**, θ_i , from a training set
- Each weight $\theta_i \in \mathbb{R}$, is associated with one of the input features x_i
- The weight θ_i represents how important x_i is to the classification decision
 - can be positive (meaning the feature is associated with the class)
 - or negative (meaning the feature is not associated with the class)
- E.g. “positive sentiment” versus “negative sentiment”
 - The features represent counts of words in a document
 - $P(y = 1|x)$ is the probability that the document has positive sentiment
 - $P(y = 0|x)$ is the probability that the document has negative sentiment.
 - “*awesome*” has a high positive weight
 - “*abysmal*” has a very negative weight

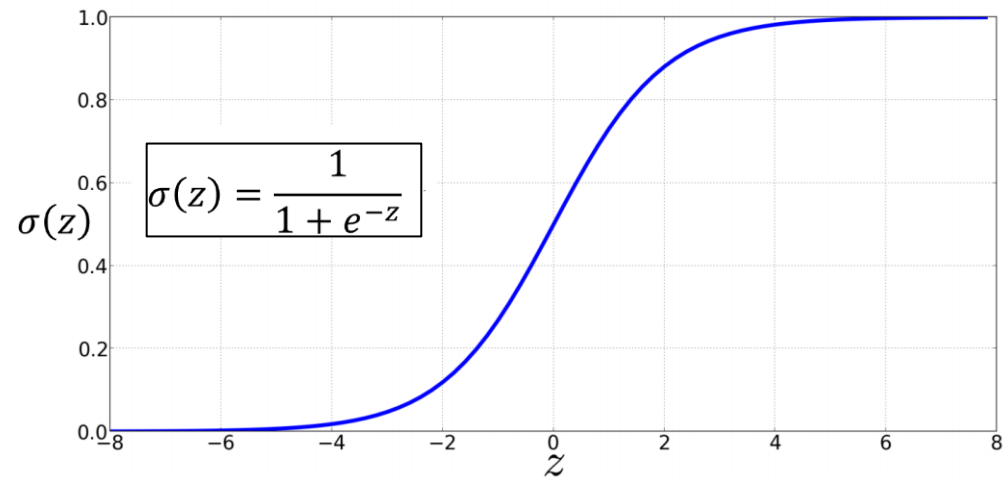
$$h_{\theta}(x) = \theta^T x$$

“Squishing” results to be between 0 and 1

- $h_{\theta}(x)$ is not forced to be a legal probability, that is, to lie between 0 and 1
 - In fact, since weights are real-valued, the output might even be negative
 - $h_{\theta}(x)$ ranges from $-\infty$ to ∞ .
- To create a probability, we'll pass $h_{\theta}(x)$ through the **sigmoid** function, $\sigma(z)$.
 - The sigmoid function (named because it looks like an s) is also called the **logistic function**.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma(h_{\theta}(x)) = \frac{1}{1 + e^{-h_{\theta}(x)}}$$

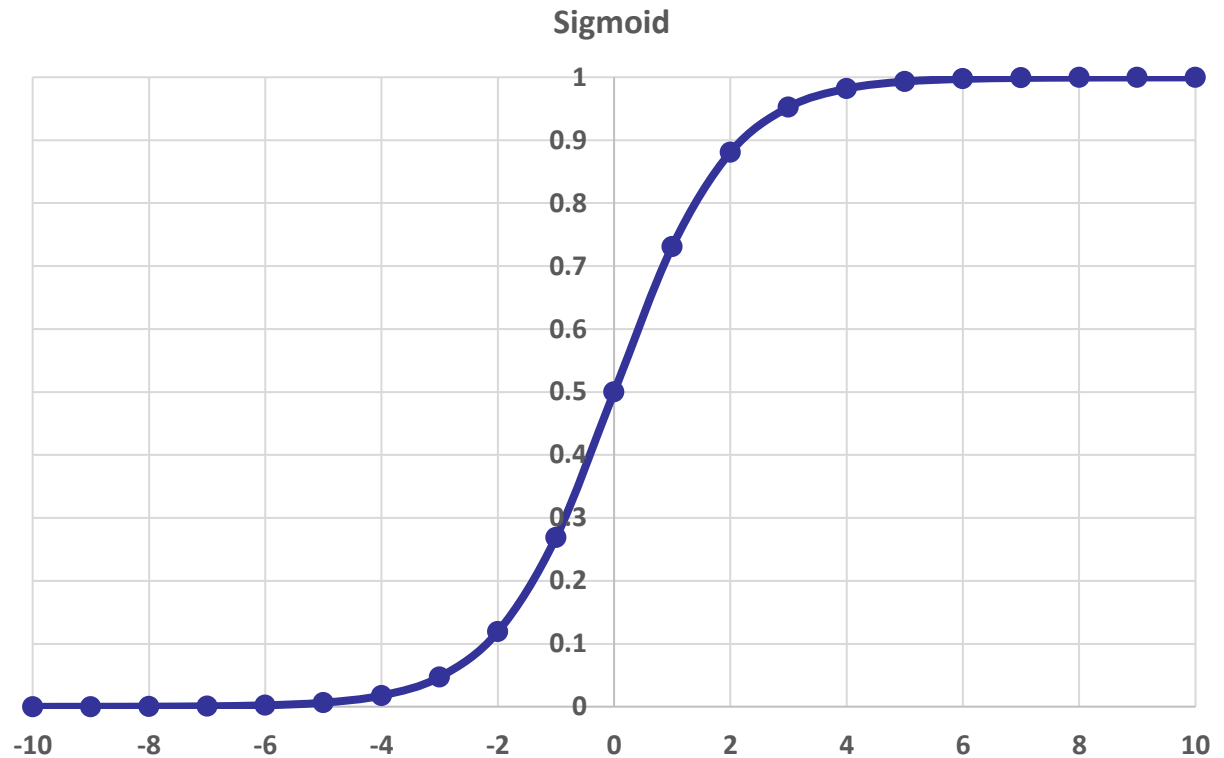


$$z = \theta^T x + b$$

$$h(x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

The Sigmoid

x	Sigmoid
-10	4.53979E-05
-9	0.000123395
-8	0.00033535
-7	0.000911051
-6	0.002472623
-5	0.006692851
-4	0.01798621
-3	0.047425873
-2	0.119202922
-1	0.268941421
0	0.5
1	0.731058579
2	0.880797078
3	0.952574127
4	0.98201379
5	0.993307149
6	0.997527377
7	0.999088949
8	0.99966465
9	0.999876605
10	0.999954602



Advantages of a Sigmoid

- Maps real-valued numbers ($\mathbb{R}$) into the range $[0,1]$
- Nearly linear around 0 but has a sharp slope toward the ends, it tends to squash outlier values toward 0 or 1
- It is differentiable, which is be handy for learning
- To make it a probability, we just need to make sure that the two cases, $p(y = 1)$ and $p(y = 0)$, sum to 1:

$$P(y = 1) = \sigma(\theta^T x)$$

$$= \frac{1}{1+e^{-\theta^T x}}$$

$$P(y = 0) = 1 - \sigma(\theta^T x)$$

$$= 1 - \frac{1}{1+e^{-\theta^T x}}$$

$$= \frac{e^{-\theta^T x}}{1+e^{-\theta^T x}}$$

- How do we decide?
 - For a test instance x , we say **yes** if the probability $P(y = 1|x)$ is more than .5, and **no** otherwise.
 - We call .5 the **decision boundary**

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

How about other thresholds?

Logistic Regression Decision Boundary

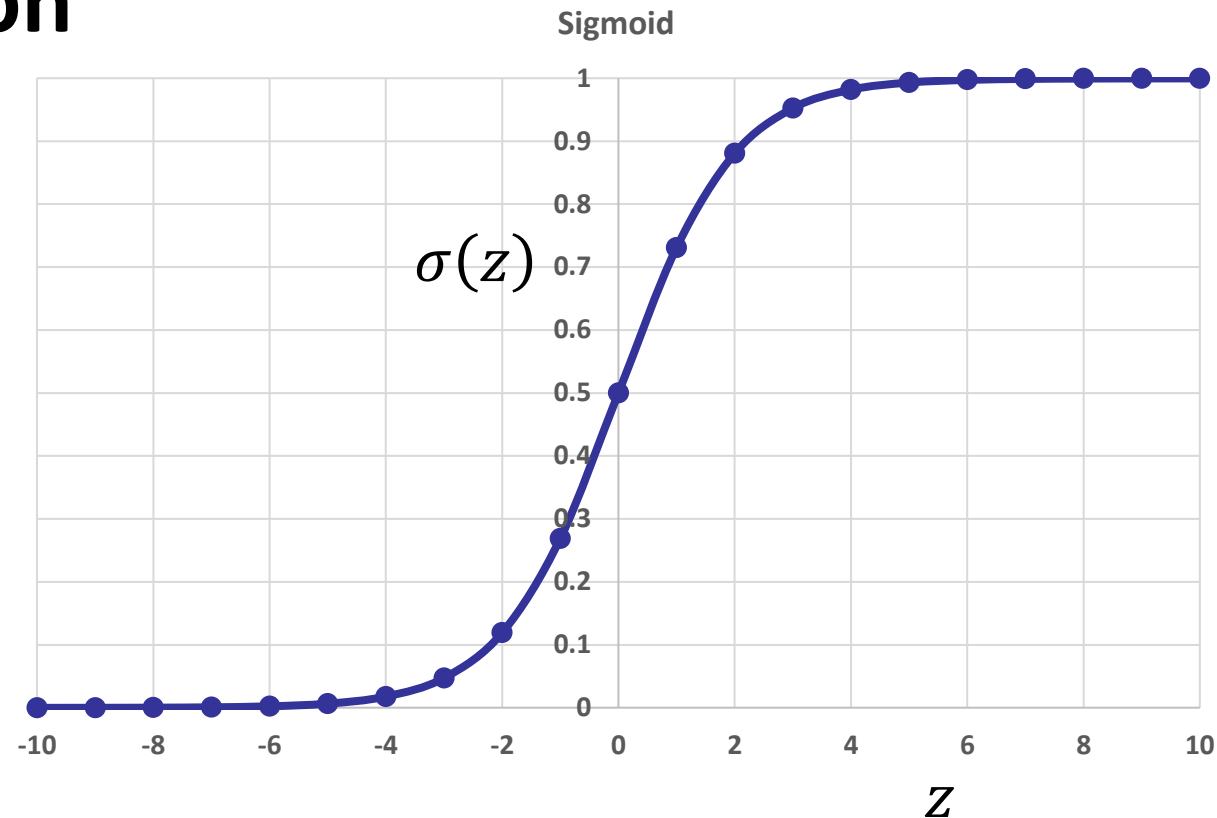
Agha Ali Raza



Logistic regression

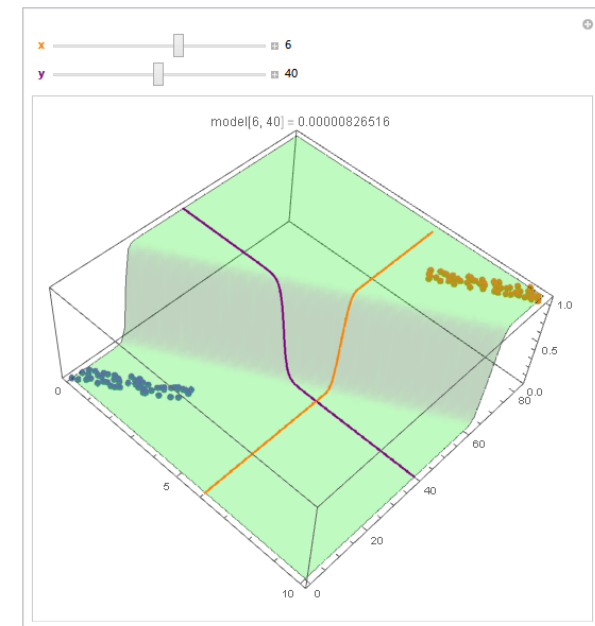
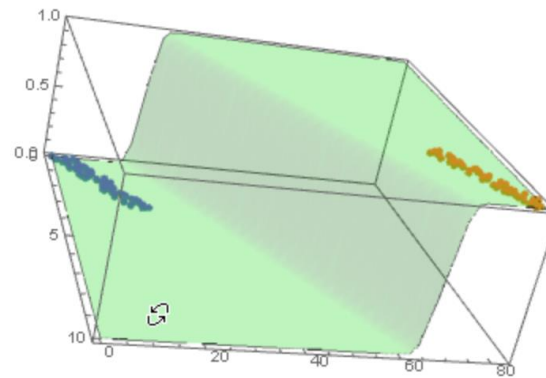
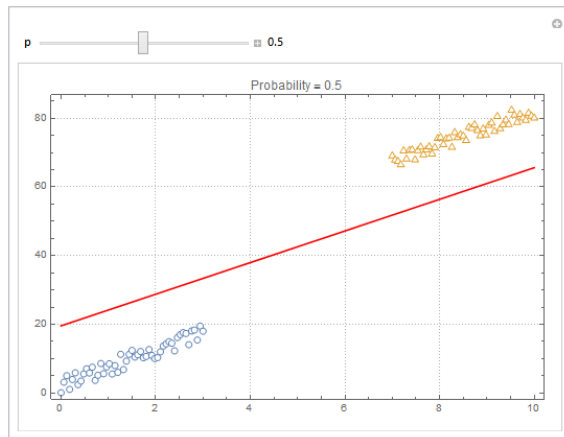
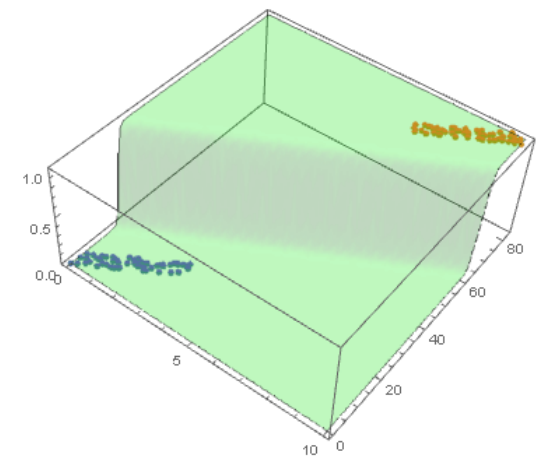
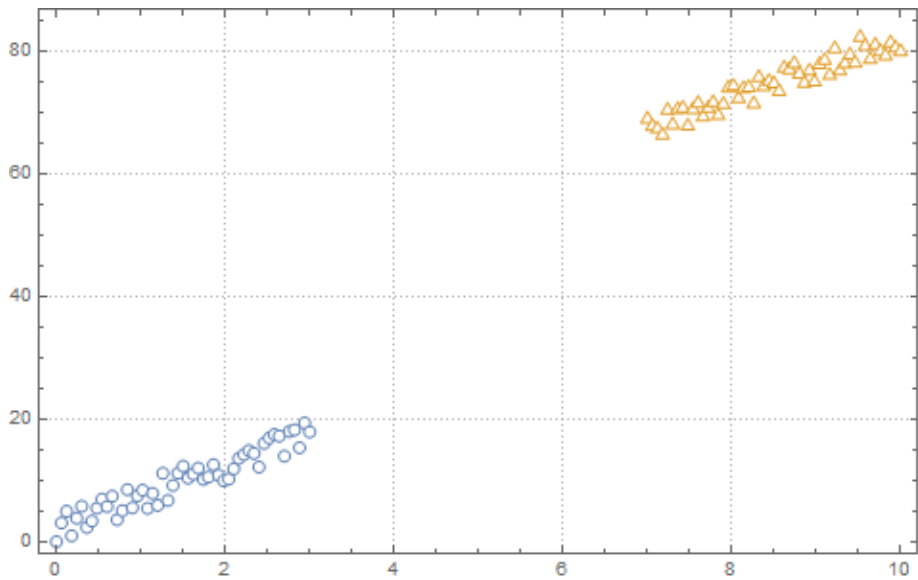
$$z = \theta^T X$$

$$h(x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

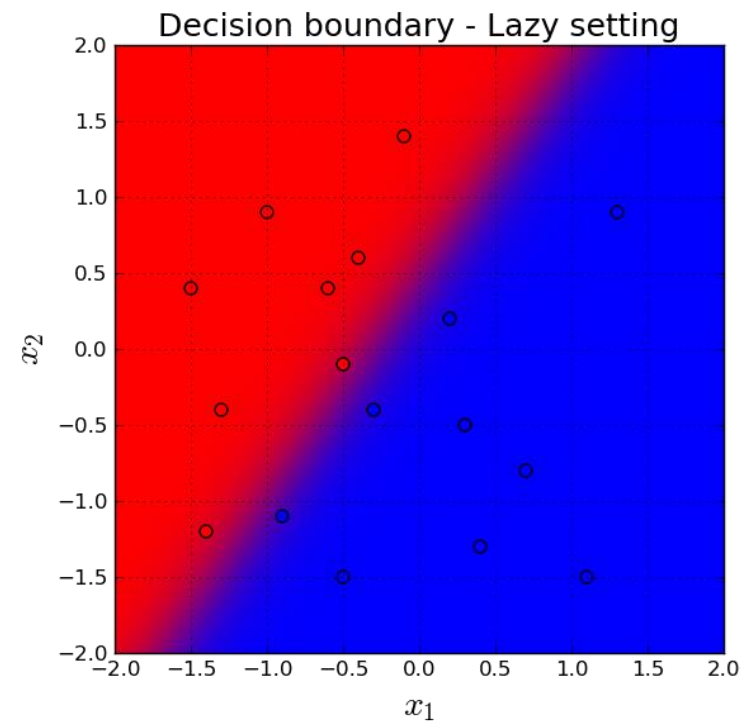
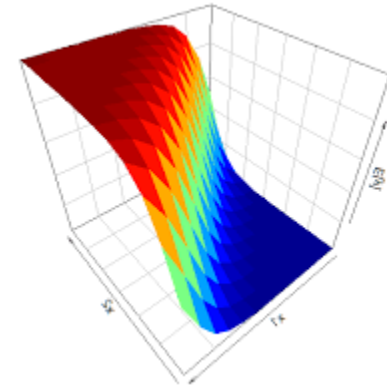
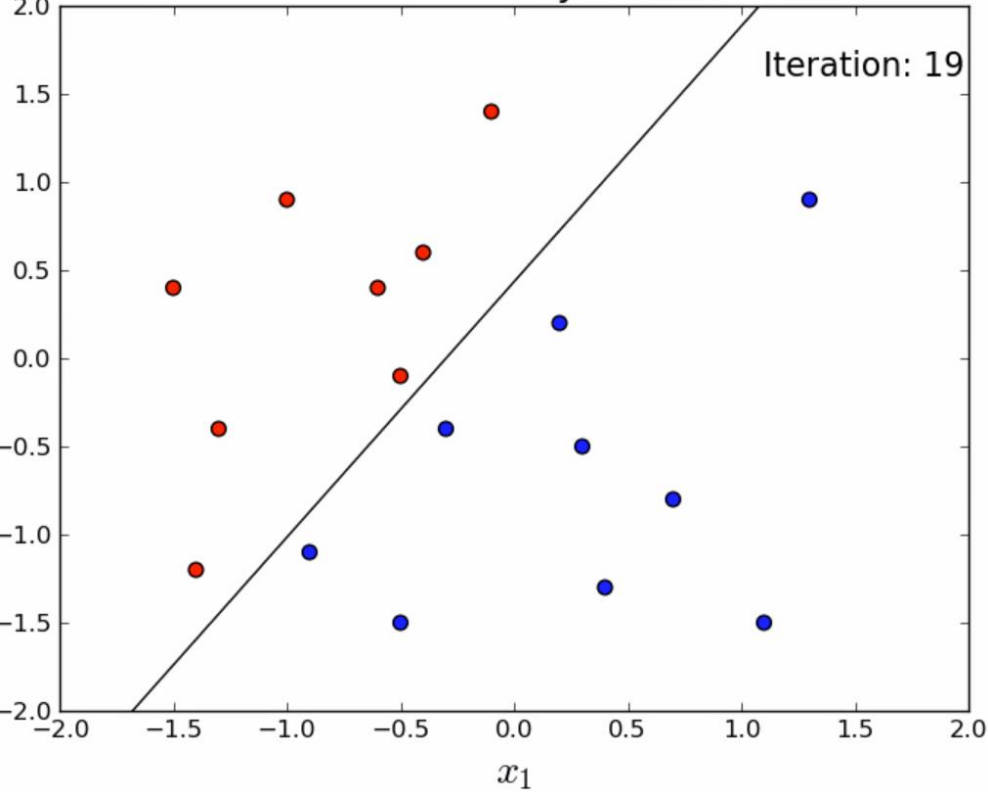


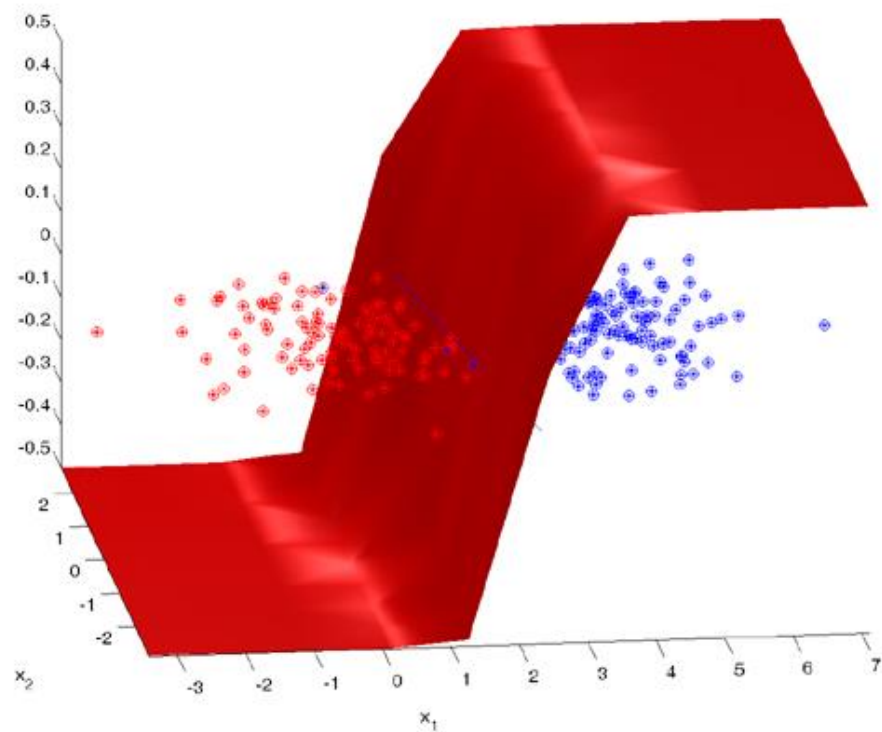
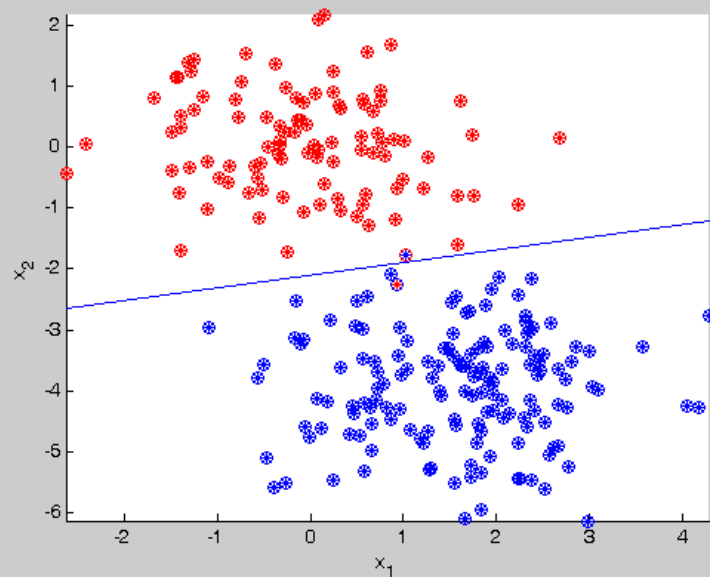
Suppose, predict “y=1” if $\sigma(z) \geq 0.5$
i.e. $\theta^T X \geq 0$

predict “y=0” if $\sigma(z) < 0.5$
i.e. $\theta^T X < 0$

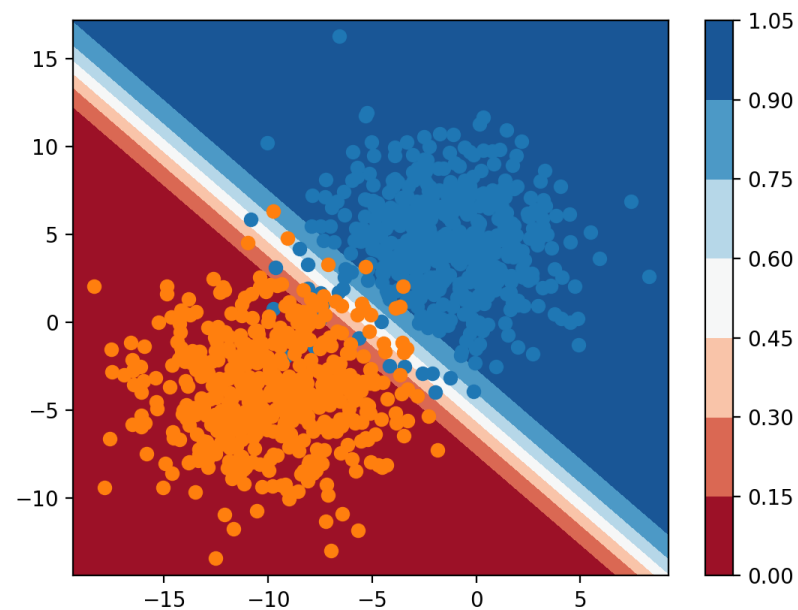
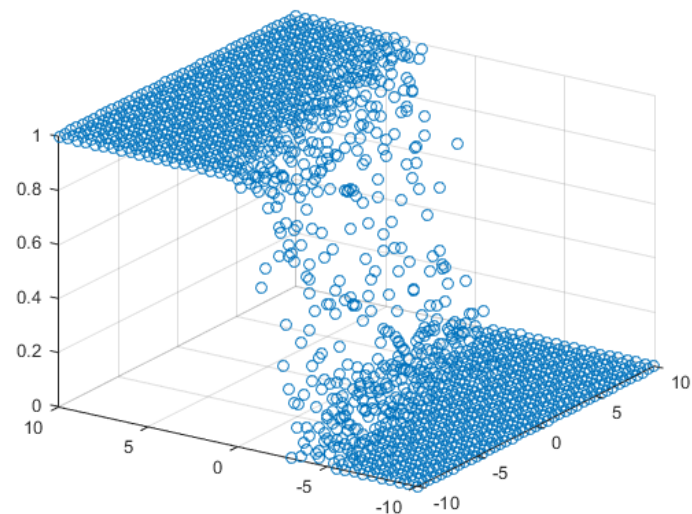
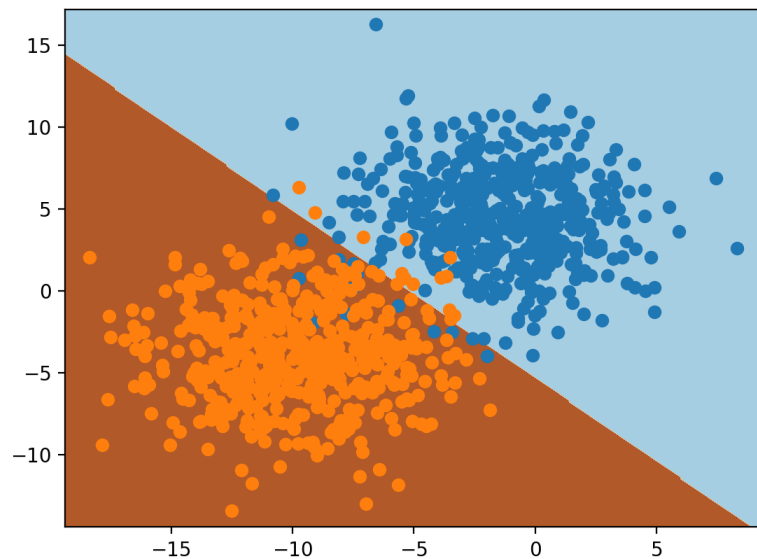


<https://datascience.stackexchange.com/questions/49573/how-to-plot-logistic-regression-decision-boundary>





<http://strijev.com/sources/demoDataGen.php>



<https://machinelearningmastery.com/plot-a-decision-surface-for-machine-learning/>
<https://stackoverflow.com/questions/29360872/fitting-3d-sigmoid-to-data>

Logistic Regression Cost Function

Agha Ali Raza



Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$

m examples $x \in \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_n \end{bmatrix} \quad x_0 = 1, y \in \{0, 1\}$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

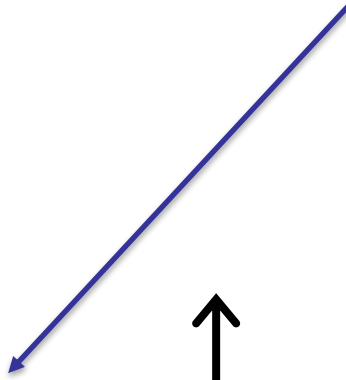
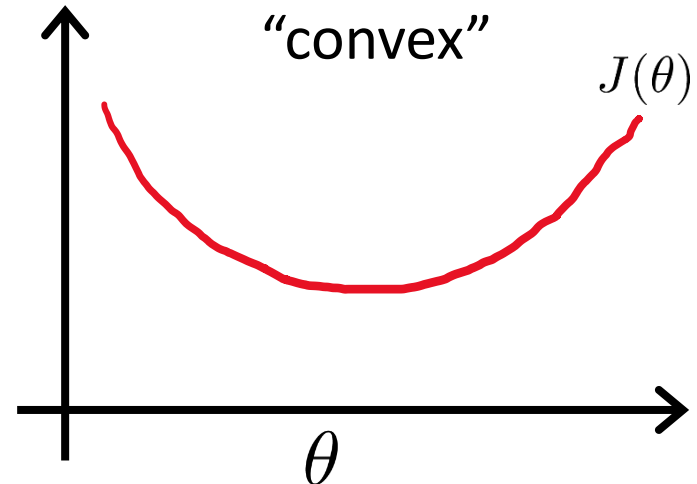
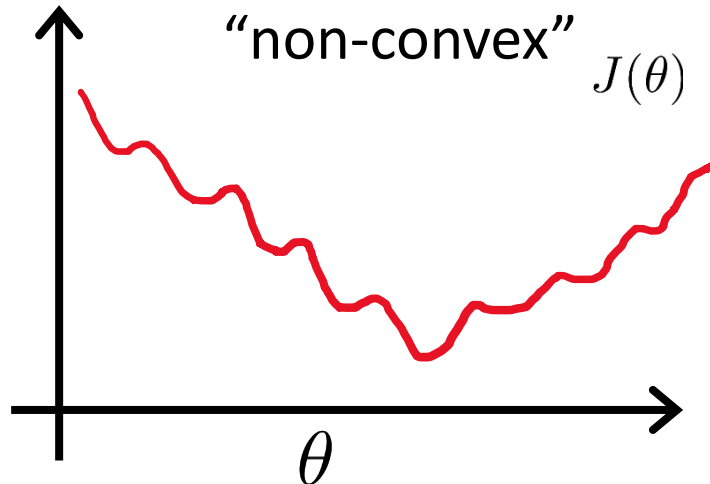
How to choose parameters θ ?

Cost function

$$\text{cost}(h_{\theta}(x), y)$$

Linear regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^m \boxed{\frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2}$

$$\text{cost}(h_{\theta}(x), y) = \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2} \left(\frac{1}{1 + e^{-\theta^T x^{(i)}}} - y^{(i)} \right)^2$$



Our goals for the cost function

- Differentiable
- Convex

$$h_{\theta}(x) = \frac{1}{1+e^{-\theta^T x^{(i)}}} \text{ varies between 0 and 1.}$$

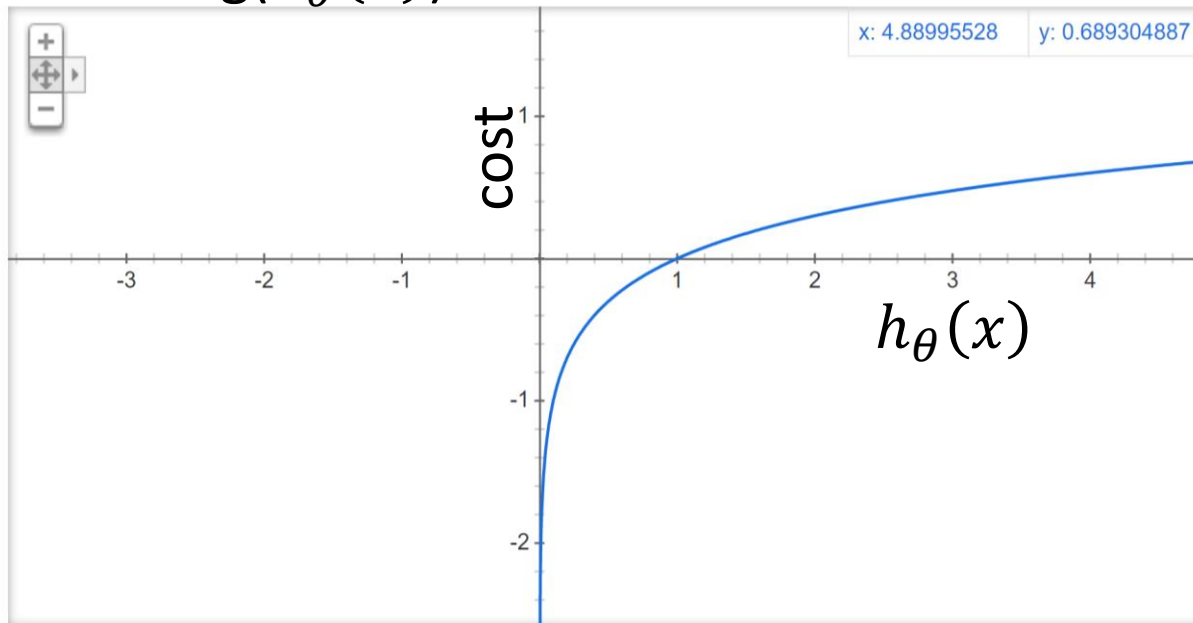
- Punish incorrect answers with high cost
 - **y=1**, $h_{\theta}(x) \rightarrow 0$
 - **y=0**, $h_{\theta}(x) \rightarrow 1$
- Reward correct answers with a low cost
 - **y=1**, $h_{\theta}(x) \rightarrow 1$
 - **y=0**, $h_{\theta}(x) \rightarrow 0$

Plot it live: <https://www.desmos.com/calculator>

- Rewards $y = 1$, $h_{\theta}(x) = 1$, with cost = 0
- Rewards $y = 1$, $h_{\theta}(x) \rightarrow 0$ even further with asymptotically smaller costs

$y=1$

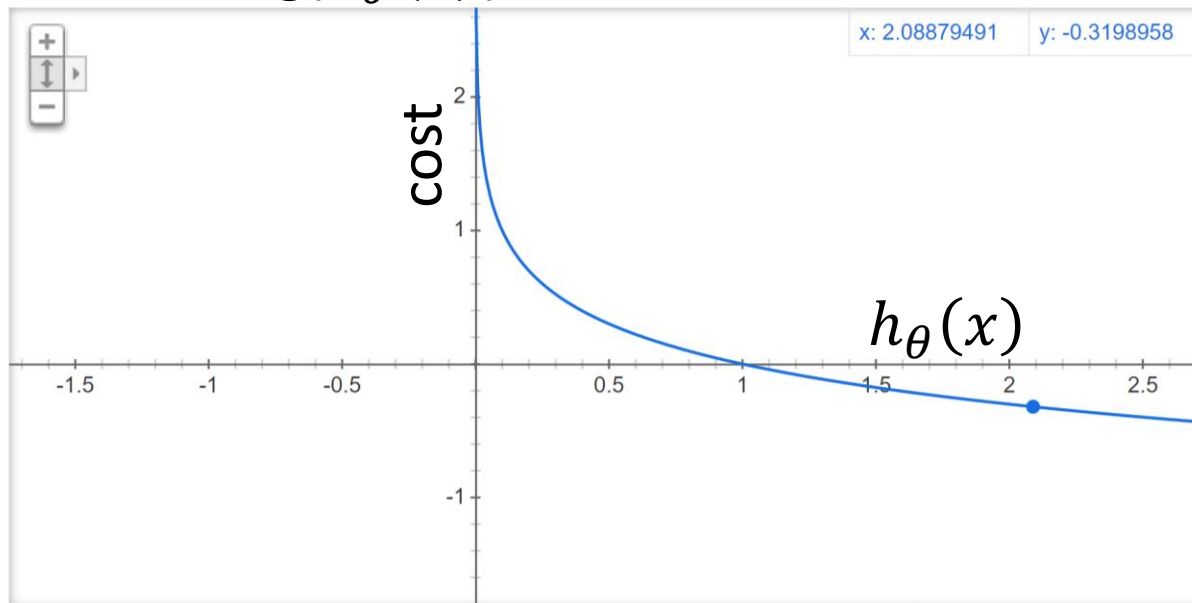
cost = $\log(h_{\theta}(x))$



- Rewards $y = 1, h_{\theta}(x) = 1$, with cost = 0
- Punishes $y = 1, h_{\theta}(x) \rightarrow 0$ with asymptotically higher costs

$y=1$

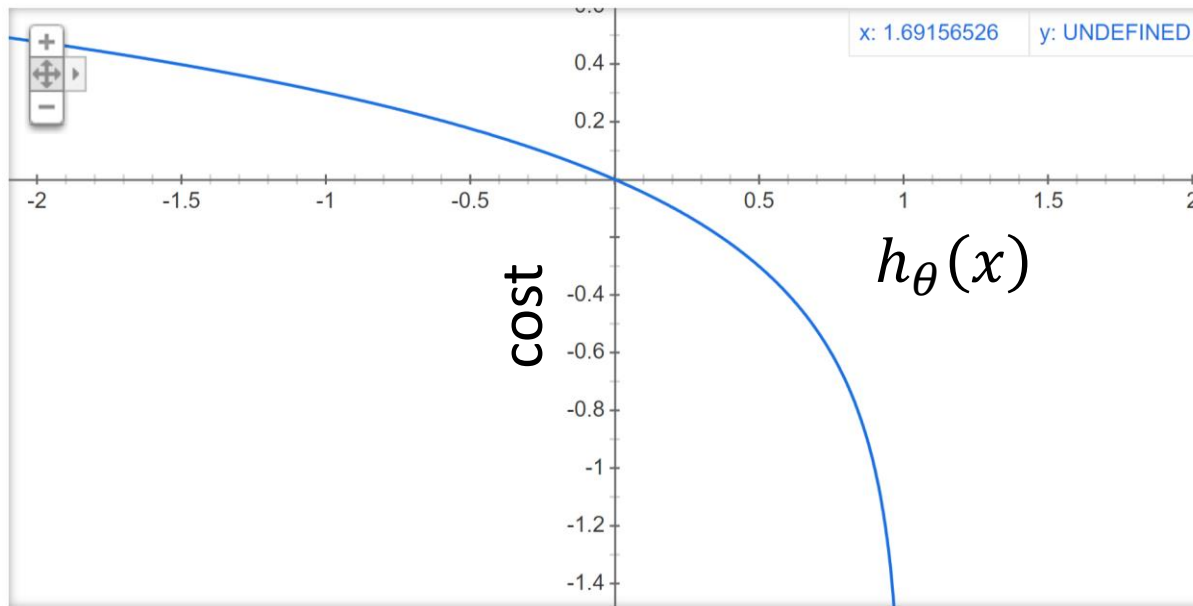
$$\text{cost} = -\log(h_{\theta}(x))$$



- Rewards $y = 0$, $h_{\theta}(x) = 0$, with cost = 0
- Rewards $y = 0$, $h_{\theta}(x) \rightarrow 1$ even further with asymptotically smaller costs

$y=0$

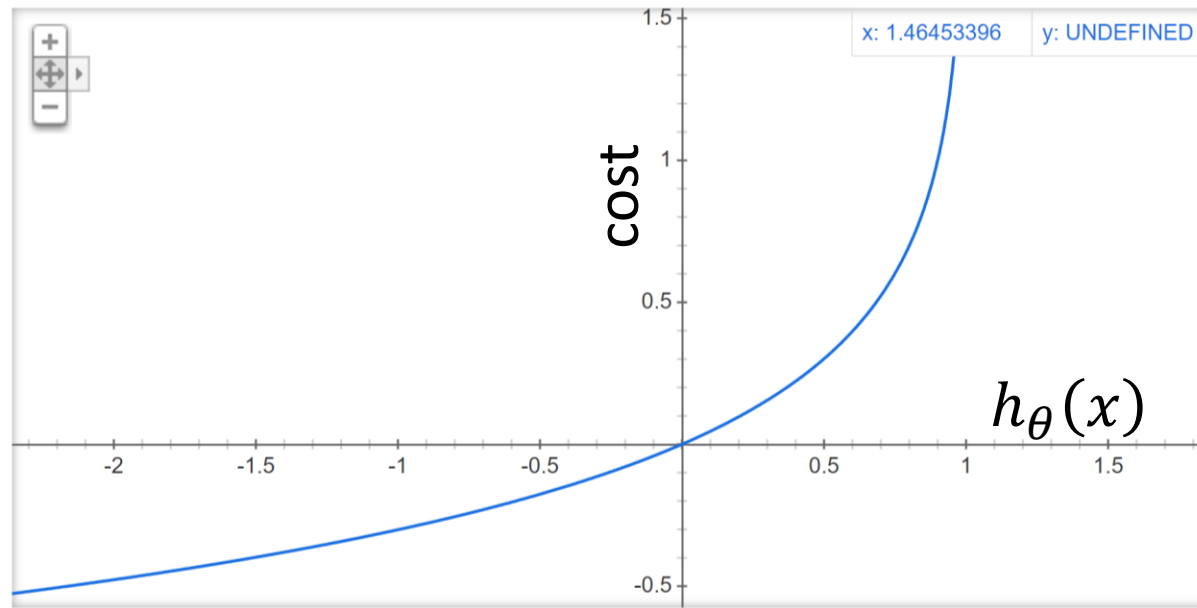
$$\text{cost} = \log(1 - h_{\theta}(x))$$



- Rewards $y = 0, h_{\theta}(x) = 0$, with cost = 0
- Punishes $y = 0, h_{\theta}(x) \rightarrow 1$ with asymptotically higher costs

y=0

$$\text{cost} = -\log(1 - h_{\theta}(x))$$



Logistic regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

Can we combine these and create a switch that turns on the correct cost values for the values of y ?

Note: $y = 0$ or 1 always (Bernoulli Distribution)

We can combine summations and products using interpolation:

e.g., if $y \in \{0,1\}$

- $\text{Cost} = y \cdot a + (1 - y) \cdot b$, chooses a if $y=1$ and b if $y=0$
- $\text{Cost} = a^y b^{(1-y)}$, chooses a if $y=1$ and b if $y=0$

Logistic regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

Can we combine these and create a switch that turns on the correct cost values for the values of y ?

Note: $y = 0$ or 1 always (Bernoulli Distribution)

$$\text{cost}(\mathbf{h}_{\theta}(\mathbf{x}), \mathbf{y}) = -\mathbf{y} \log(\mathbf{h}_{\theta}(\mathbf{x})) - (1 - \mathbf{y}) \log(1 - \mathbf{h}_{\theta}(\mathbf{x}))$$

$$\text{if } y = 1: \text{cost}(h_{\theta}(x), y) = -\log(h_{\theta}(x))$$

$$\text{if } y = 0: \text{cost}(h_{\theta}(x), y) = -\log(1 - h_{\theta}(x))$$

$$\text{cost}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

- We have created a loss function that prefers the correct class labels of the **training examples** to be more likely
 - It **estimates** the model parameters to **maximize** the **likelihood** of the training set
 - This is called **maximum likelihood estimation** (MLE)
- We choose the parameters θ_i that maximize the log probability of the true y labels in the training data given the observations x .
- The resulting loss function is the negative log likelihood loss, generally called the **cross-entropy loss**.

Logistic regression cost function

$$\begin{aligned} J(\theta) &= \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)}) \\ &= -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right] \end{aligned}$$

To fit parameters θ :

$$\min_{\theta} J(\theta)$$

To make a prediction given new x :

$$\text{Output } h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}} \quad \text{gives } p(y = 1|x; \theta)$$



LUMS



Logistic Regression

Gradient Descent

Agha Ali Raza



CS535/EE514 – Machine Learning

Batch Gradient Descent

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

}

(simultaneously update all θ_j)

$$\frac{\partial}{\partial \theta_j} (J(\theta)) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(see SLP3 for derivation)

Batch Gradient Descent

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

Want $\min_{\theta} J(\theta)$:

Repeat {

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

} (simultaneously update all θ_j)

Algorithm looks identical to linear regression!

Batch Gradient Descent

$$J(\theta) = y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))$$
$$\frac{\partial}{\partial \theta_j} (J(\theta)) = (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Repeat until $\theta_{t+1} - \theta_t \geq \varepsilon$ {
 $\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$
 (simultaneously update all θ_j)
}

Stochastic Gradient Descent

$$J(\theta) = y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))$$
$$\frac{\partial}{\partial \theta_j} (J(\theta)) = (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Repeat until $\theta_{t+1} - \theta_t \geq \varepsilon$ {

Repeat for each training instance $(x^{(i)}, y^{(i)})$, in random order{

$$\theta_j := \theta_j - \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update all θ_j)

}

}

Mini-batch Gradient Descent

$$J(\theta) = y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)}))$$
$$\frac{\partial}{\partial \theta_j} (J(\theta)) = (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Repeat until $\theta_{t+1} - \theta_t \geq \varepsilon$ {
 Repeat for each randomly selected minibatch of size k {

$$\theta_j := \theta_j - \alpha \frac{1}{k} \sum_{i=1}^k (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

 (simultaneously update all θ_j)
 }
}

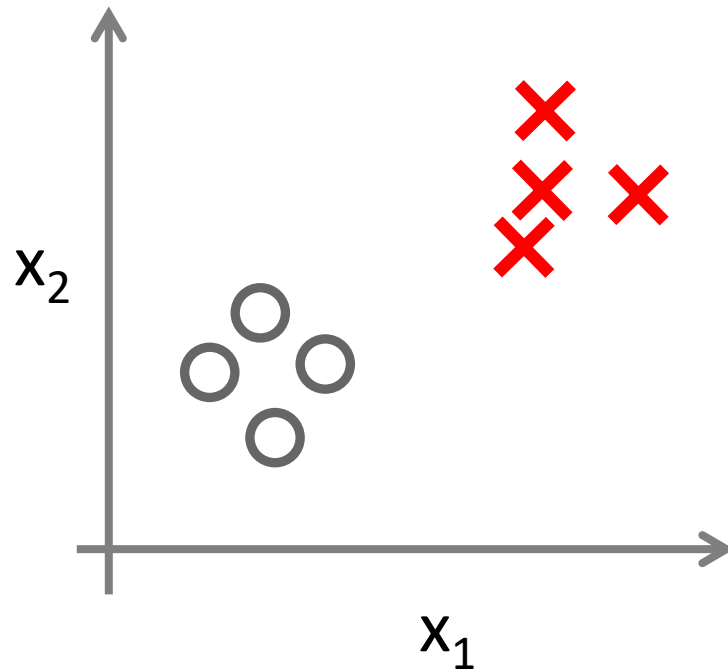
Logistic Regression

Multiclass (multinomial) classification

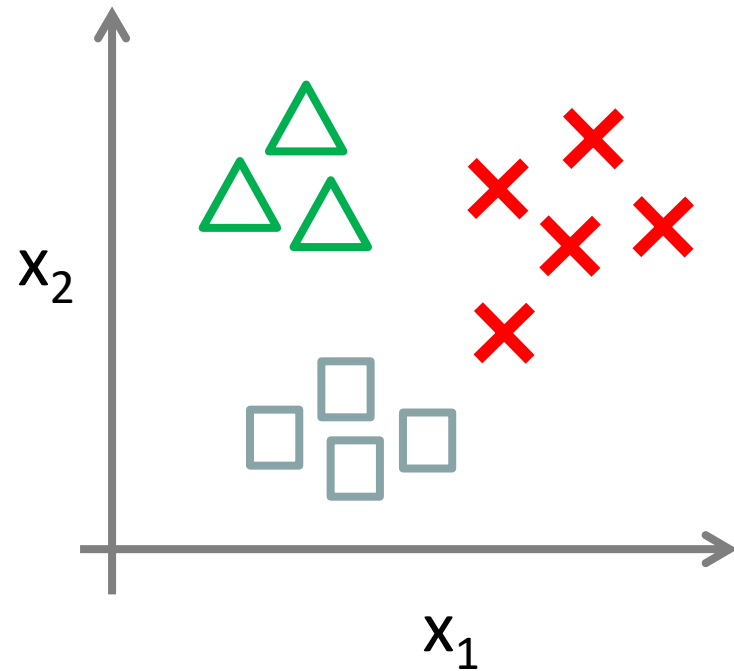
Agha Ali Raza



Binary classification:



Multi-class classification:



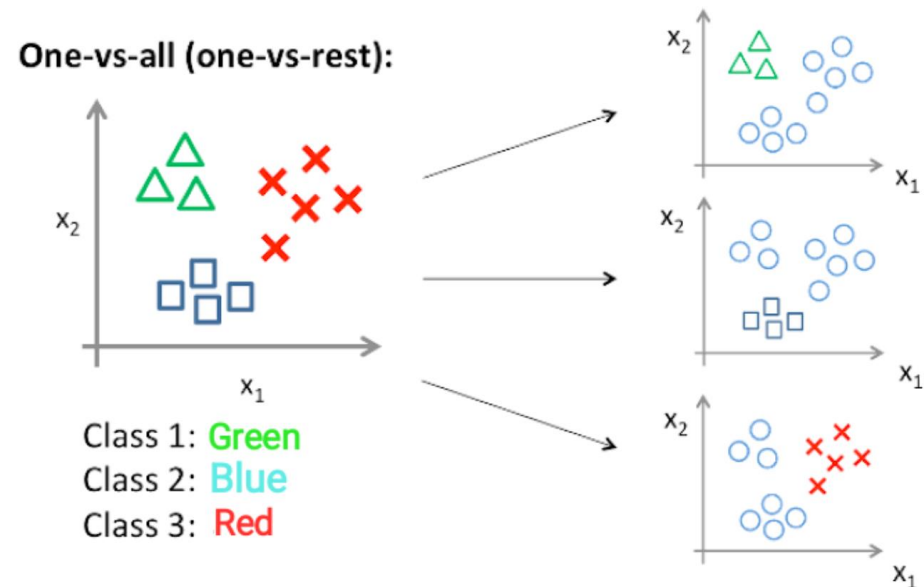
Multiclass classification

- Logistic Regression inherently models binary classification and needs meta-strategies for multiclass classification
- Multiclass classification
 - Multiple class labels are present in the dataset.
- Binary classifiers for multiclass classification:
 - One vs Rest (aka One vs All)
 - Splits a multi-class classification into one binary classification problem per class.
 - n -class instances then n binary classifier models
 - One vs One
 - Splits a multi-class classification into one binary classification problem per each pair of classes.
 - n -class instances then $\binom{n}{2} = \frac{n * (n-1)}{2}$ binary classifier models

<https://towardsdatascience.com/multi-class-classification-one-vs-all-one-vs-one-94daed32a87b>

One vs All (One vs Rest)

- One classifier per class ($x \in class_i$ vs $x \notin class_i$) for all n classes
 - In the scoring phase all the n classifiers predict the probability of each class and class with highest probability is selected
-
- Classifier 1:- [Green] vs [Red, Blue]
 - Classifier 2:- [Blue] vs [Green, Red]
 - Classifier 3:- [Red] vs [Blue, Green]



Create separate data sets for training

Main Dataset

Features			Classes
x1	x2	x3	G
x4	x5	x6	B
x7	x8	x9	R
x10	x11	x12	G
x13	x14	x15	B
x16	x17	x18	R

Training Dataset 1
Class :- Green

Features			Green
x1	x2	x3	+1
x4	x5	x6	-1
x7	x8	x9	-1
x10	x11	x12	+1
x13	x14	x15	-1
x16	x17	x18	-1

Class 1 :- Green Class 2 :- Blue Class 3 :- Red

Training Dataset 2
Class :- Blue

Features			Blue
x1	x2	x3	-1
x4	x5	x6	+1
x7	x8	x9	-1
x10	x11	x12	-1
x13	x14	x15	+1
x16	x17	x18	-1

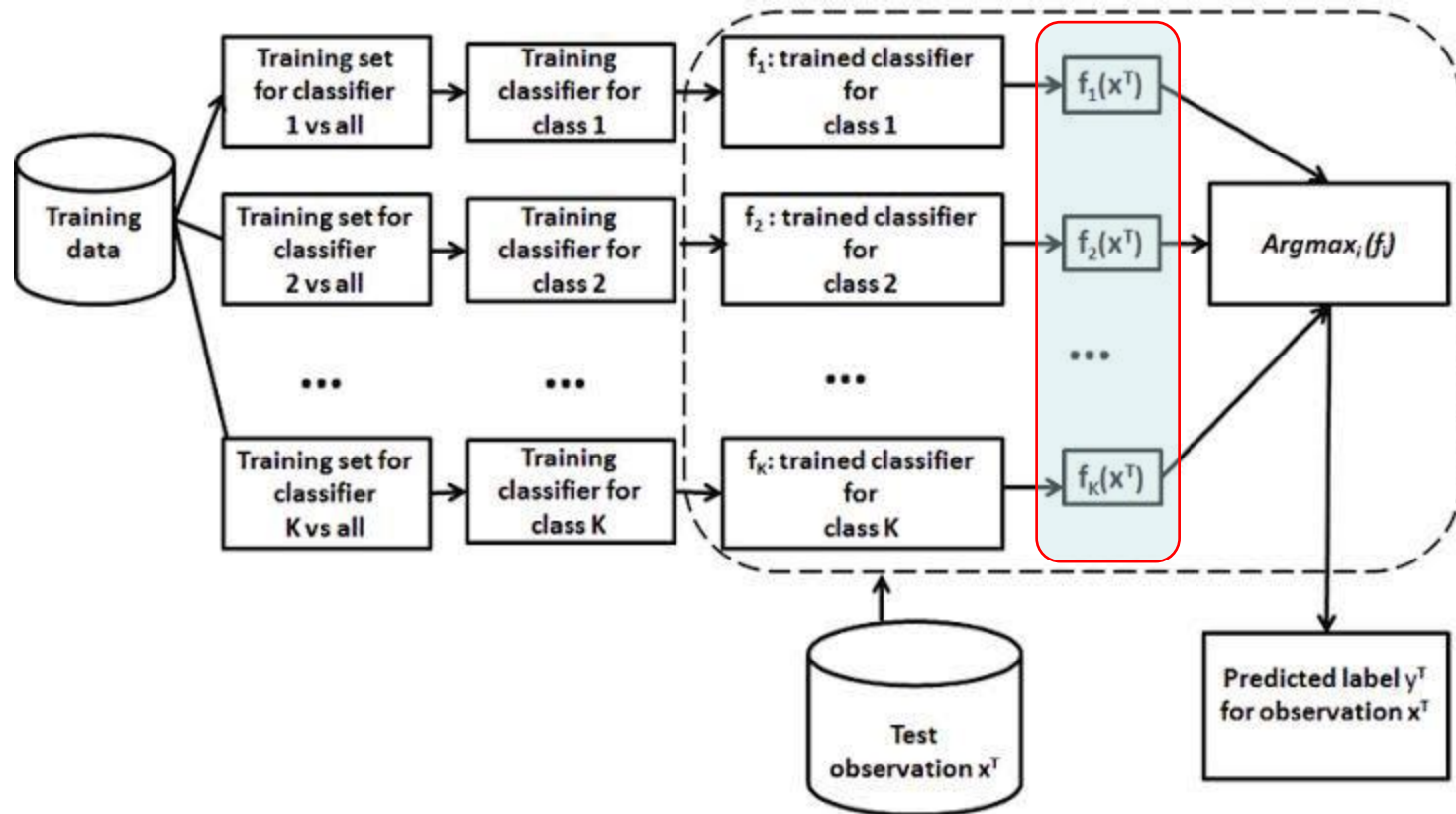
Training Dataset 3
Class :- Red

Features			Red
x1	x2	x3	-1
x4	x5	x6	-1
x7	x8	x9	+1
x10	x11	x12	-1
x13	x14	x15	-1
x16	x17	x18	+1

<https://towardsdatascience.com/multi-class-classification-one-vs-all-one-vs-one-94daed32a87b>

One vs All pipeline

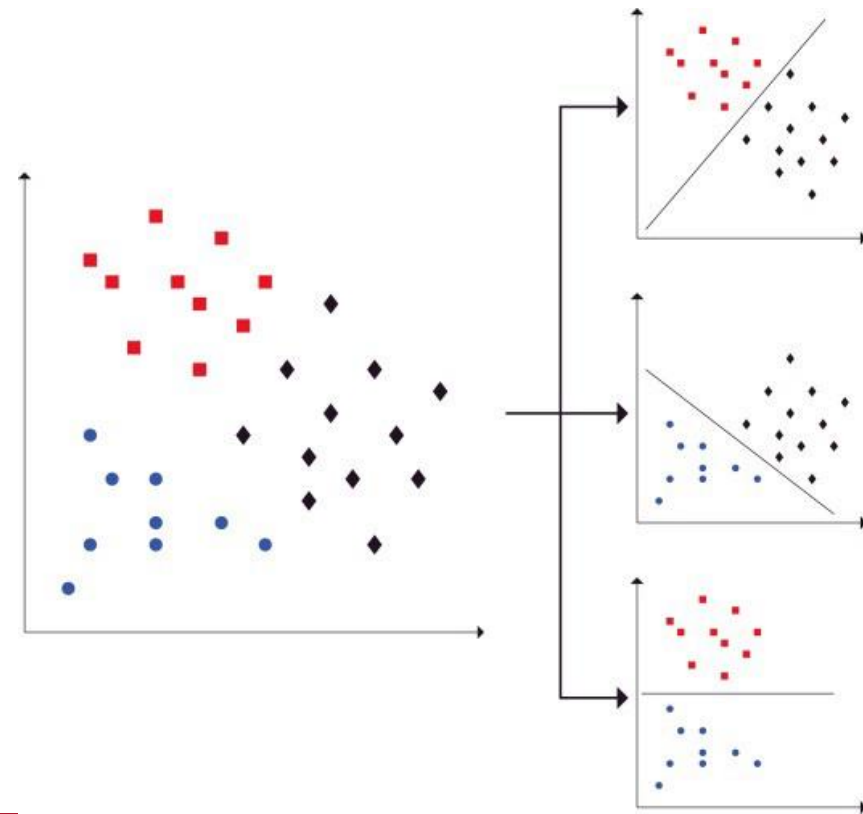
- Each of these is a probability value between 0 and 1.
- These probabilities do not need to add up to 1.



<https://towardsdatascience.com/multi-class-classification-one-vs-all-one-vs-one-94daed32a87b>

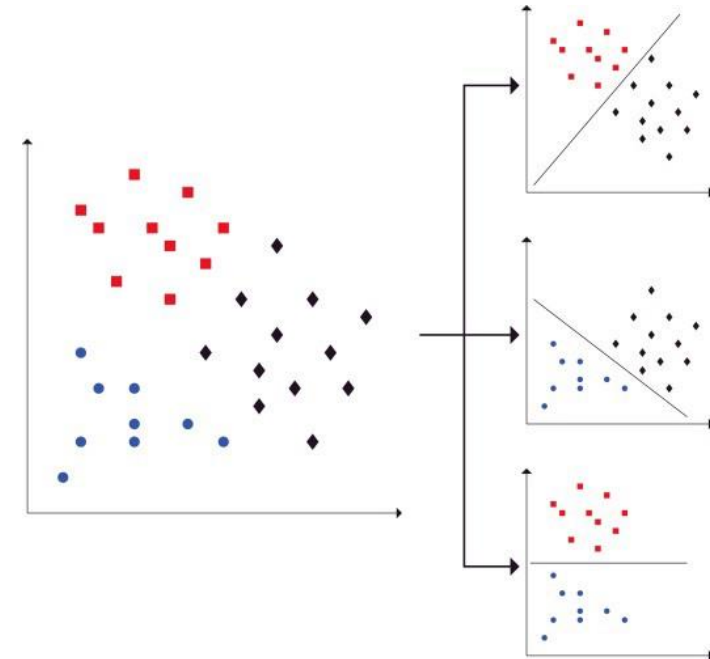
One vs One

- For n classes, we must generate $\frac{N * (N-1)}{2}$ binary classifier models
- Using this classification approach, we split the primary dataset into one dataset for each class vs every other class.
- Classifier 1: Black vs. Blue
- Classifier 2: Black vs. Red
- Classifier 3: Blue vs. Red



One vs One

- Each binary classifier predicts one class label and the model with the most predictions or votes is predicted by the one-vs-one strategy.
- If the binary classification models predict a numerical class membership, such as a probability, then the *argmax* of the sum of the scores (class with the largest sum score) is predicted as the class label.
 - These probabilities for each class do not need to add up to 1, so these sums are “scores”
 - $\text{Argmax}(\text{Sum of Pr}(\text{red}), \text{Sum of Pr}(\text{blue}), \text{Sum of Pr}(\text{green}))$



oVr vs oVo

- oVr trains fewer classifiers and is faster overall
 - But the classes are imbalanced e.g. red vs (blue + green)
- oVo is less prone to imbalance in datasets
 - All classes are balanced, if the original dataset was balanced e.g. red vs green, red vs. blue,...

Applicability

- We can use these techniques with any binary classifier that can return scores or probabilities
- We can also tweak these techniques for returning multiple labels – in case of multilabel classification e.g. emotions

Logistic Regression Softmax

Agha Ali Raza



Softmax

- Recall that the sigmoid produces a number between 0 and 1.
 - The sum of the probabilities of an email being either spam or not spam is 1.0.
 - But OvR, and OvO do not abide by these rules for the sums of probabilities across n classes
- Softmax extends this idea into a multi-class world.
 - Softmax assigns a probability to each class in a multi-class problem and the probabilities across classes add up to 1.0.
 - Works only when each example is a member of only one class (i.e., it does not work directly for multilabel)
 - If this is not true, rely on multiple logistic regression

A quick side note on math

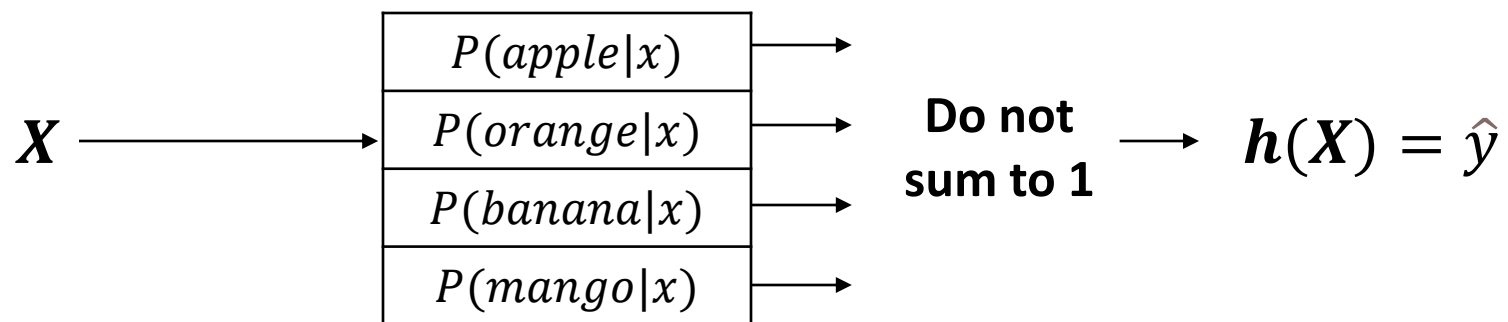
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$= \frac{1}{1 + \frac{1}{e^x}} \qquad = \frac{1}{\frac{(e^x + 1)}{e^x}}$$

$$= \frac{e^x}{e^x + 1} \qquad = \frac{e^x}{e^x + e^0}$$

Generalizing Logistic Regression

- For multiple classes ($\# \text{classes} > 2$) we can generalize logistic regression to Softmax regression
 - One-of- C classes
 - E.g. 4 classes (0, 1, 2, 3)



- Softmax converts these C raw scores to probabilities that sum to 1.

Softmax activation function

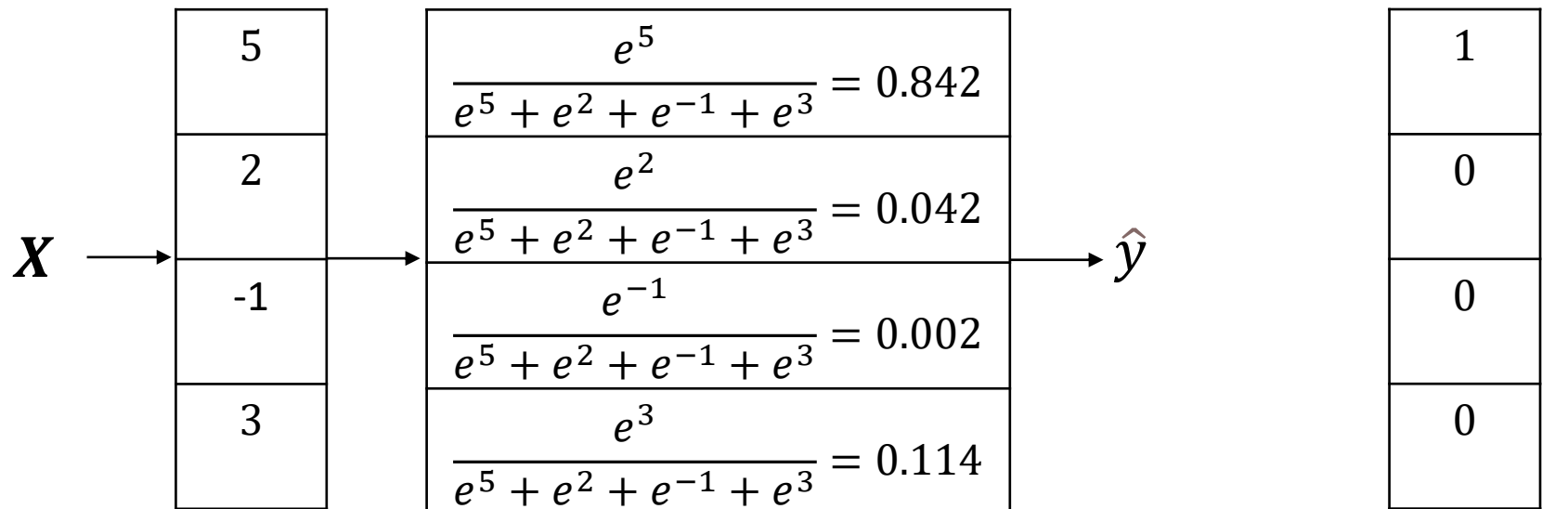
$$z = wx + b$$

$$S(z)_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} = \frac{e^{z_i}}{e^{z_1} + e^{z_2} + \dots + e^{z_C}}$$

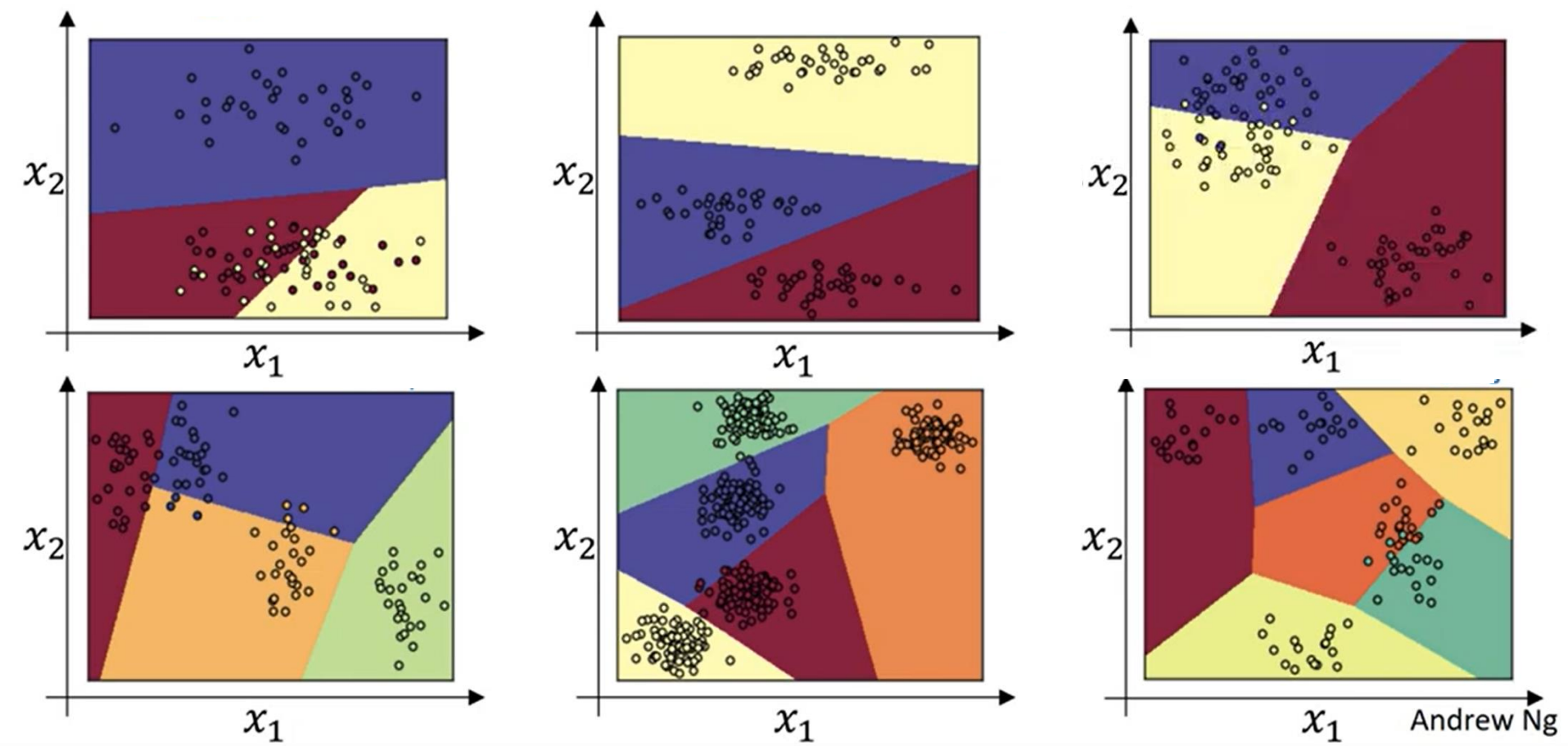
For 2 classes $\{z, 0\}$, the softmax is identical to sigmoid

$$S(z)_1 = \frac{e^z}{e^z + e^0} = \frac{e^z}{e^z + 1} = \sigma(z)$$

$$S(z)_2 = \frac{e^0}{e^z + e^0} = \frac{1}{e^z + 1} = 1 - \sigma(z)$$



Softmax examples



Loss Function

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \hat{y} = \begin{bmatrix} 0.3 \\ 0.2 \\ 0.1 \\ 0.4 \end{bmatrix}$$

$$L(y, \hat{y}) = - \sum_{j=1}^c y_j \log \hat{y}_j$$

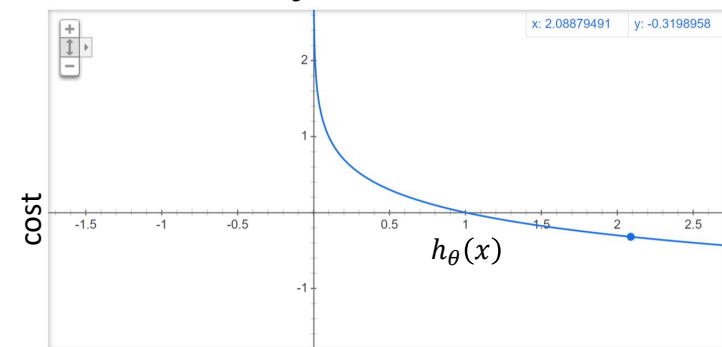
$$L(y, \hat{y}) = -(0. \log \hat{y}_1 + 1. \log \hat{y}_2 + 0. \log \hat{y}_3 + 0. \log \hat{y}_4)$$

$$L(y, \hat{y}) = -y_2 \log \hat{y}_2 = -\log \hat{y}_2$$

$$J(y, \hat{y}) = \frac{1}{m} \sum_{i=1}^m L(y^{(i)}, \hat{y}^{(i)})$$

y=1

cost = $-\log(h_{\theta}(x))$



For more details please visit
<http://aghaaliraza.com>



Thank you!